
第 19 章

图形界面与 App Designer

MATLAB 高级编程与工程应用

本章目录

- §19.1 图形界面入门
- §19.2 回调与数据共享
- §19.3 信号时域/频谱查看器

本章环境卡（系统提示词）

用户正在学习信号与系统和 MATLAB 仿真，以下要求适用于本次会话里所有 MATLAB 代码请求：

- 兼容 MATLAB R2023b，不使用之后版本独有的函数；
- 只返回完整、可直接运行的单文件代码 + 必要的中文注释，代码之外不附任何解释文字；
- 变量名沿用我每次指定的名称，不要改名；
- 显示符号表达式时用 `disp`，不要用 `pretty`；
- 图形界面一律用纯代码 `uifigure` 及其组件（不用已弃的 `GUIDE/.fig`，也不用 `App Designer/.mlapp`）；
- 回调可用匿名函数、嵌套函数、或放在文件最末尾的局部函数；切勿把局部函数写在脚本中段（会报“函数定义必须在文件结尾”）；
- `uiaxes` 中的图：文字清晰可读、曲线线宽 2、`grid on`（GUI 坐标轴较小，字号不必死守 24pt）；
- 不要添加任何测试桩、自检打印或调试残留。

本章导引：把信号工具装进可交互的界面

- 前面几章写的都是“运行后输出结果”的脚本。图形界面（GUI）是另一种形态：把你已经会的信号处理代码（生成信号、求频谱）包装成可交互的工具：拖滑块改频率、点按钮换信号，图随手而动。
- 对大模型辅助编程，GUI 有个关键好处：现代 MATLAB 用纯代码 `uifigure` 建界面，整个界面是一段文本代码，模型能整段输出、复制即运行；可视化的 `.fig`、`.mlapp` 工程文件则没法从对话框粘贴运行。所以本章只用纯代码 `uifigure`。
- 验证的方式要变：脚本看一个输出数，界面没有单一的“输出数”。判断一个界面对不对靠两条：一是交互行为对不对（拖滑块波形该变、按按钮该重画）；二是界面里显示的结果仍用信号知识查验。
- 还有一层：GUI 的错（回调放错位置、控件间数据没共享对、用错事件）往往不报错、界面照样运行，所以这一章要以代码核验为主。

§19.1 图形界面是什么：窗口 + 组件

- 图形界面 (GUI) 就是一个窗口，里面摆着若干组件，用户用鼠标、键盘直接操作，不必在命令行敲指令。MATLAB 里这个窗口由 `uifigure` 创建，组件都用代码生成、再放进窗口。
- 组件分几类，先知道有哪些：
 - 控件 (供操作)：按钮 `uibutton`、滑块 `uislider`、下拉框 `uidropdown`、勾选框 `uicheckbox`、输入框 `uieditfield`；
 - 显示 (供查看)：坐标轴 `uiaxes` (画曲线或图像)、文字标签 `uilabel`、图片 `uiimage`、表格 `uitable`；
 - 布局 (摆位置)：网格布局 `uigridlayout` 等，按行列把组件排整齐。
- 每个组件都是一个对象，带一批属性 (位置、大小、文字、当前值...) 和事件 (被点击、被拖动...)。后面连连看的棋盘，就是一格一个按钮、按网格布局排成的组件阵列。

§19.1 核心机制：事件驱动与消息循环

- 普通脚本从头跑到尾就结束。图形界面不是这样：组件摆好后程序不结束，而是进入一个消息循环（事件循环）停在那里等。
- 用户的每个动作（移动鼠标、点击、拖动、敲键）都被系统变成一条消息（事件）排进队列；循环每次取出一条，交给这个组件、这个事件对应的处理函数执行。这个处理函数就是回调。
- 关键点：这个循环你不用自己写。你只做两件事，创建组件、给组件的事件绑定回调（如 `btn.ButtonPushedFcn = @onPush`）；取消息、派发、再等待，由 MATLAB 在后台循环。所以一个界面程序，本质就是一组组件 + 一组回调。
- 回调之间需要交互/共享信息：连连看里先点的那一块、整个棋盘矩阵的状态，得在一次次点击（一个个事件）之间共享。

消息循环转一圈：建好界面 → 等事件 → 事件来了执行对应回调 → 回调改状态、重绘界面 → 回到等待

§19.1 App Designer: 本章为何仍用纯代码

- MATLAB 官方做界面的主力工具是 **App Designer**: 在可视化编辑器里拖拽摆放组件、设置属性, 它自动生成界面代码、搭建回调框架, 适合开发大型、正式的应用。
- 它操作的是 `.mlapp` 文件, 把界面布局与代码打包在一起, 要在 App Designer 里打开、编辑、运行。
- 本章用编码模型辅助编程, 要模型把整个程序以纯文本代码给出、从对话框复制即运行; `.mlapp` 是打包工程, 无法从对话框粘贴运行 (与 Simulink 的 `.slx` 同理)。
- 所以本章一律用纯代码 `uifigure`: 它和 App Designer 用的是同一批组件、同一套事件机制, 只是改成手写代码, 便于模型生成、便于粘贴运行。

§19.1 动手：第一个界面

- 用以上机制构造一个最简单的界面：一个 `uiaxes` 画正弦波、一个 `uislider` 调频率，拖动滑块波形实时更新。用到的就是前面的组件与回调。
- 滑块的事件有两个回调：`ValueChangingFcn` 在拖动过程中持续触发（可实时更新），`ValueChangedFcn` 在松开后才触发。要实时更新，绑定前者。
- 这一例状态很简单（只有频率一个值），回调只更新坐标轴里的曲线，还不涉及多组件之间共享状态。

例 19.1

用纯代码 `uifigure` 建立一个图形界面：一个坐标轴显示正弦波 $\sin(2\pi ft)$ 、一个滑块调节其频率 f ，拖动滑块时波形随之实时更新。

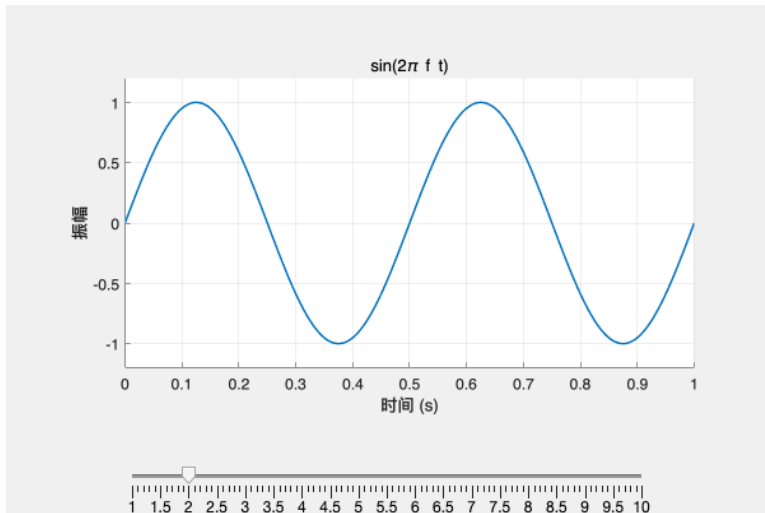
请用纯代码 `uifigure` 建一个图形界面：

放一个 `uiaxes` 显示正弦波 $\sin(2\pi f t)$ (t 取 0~1 秒)、一个 `uislider` 调节频率 f (1~10 Hz)；

拖动滑块时波形随频率实时更新；

界面创建后立即用默认频率（如 $f=2$ ）画一次初始波形。

例 19.1 运行结果·界面截图



例 19.1 核对

对结果的核验

- 初始频率 $f = 2$ 时，0 ~ 1 秒内应恰有 2 个完整周期（截图正是如此）；拖动滑块增大 f ，周期数随之增多。

对代码的核验

- 题面要求拖动时实时更新，故回调用 `ValueChangingFcn`（拖动中持续触发），而非 `ValueChangedFcn`（松开后才触发）。
- 更新波形的回调写成嵌套函数、置于主函数内部，不写在脚本中段（详见易错点）。

例 19.1 易错点

易错点：作为回调的局部函数必须放在文件末尾

- 若把回调写成的局部函数放在脚本中段，MATLAB 会报“函数定义必须在文件结尾”；这是本章最常见的报错。
- 对策：回调用匿名函数、嵌套函数（写在主函数内部），或放在文件最末尾的局部函数。本例的 `updateWave` 是嵌套函数，合规。

§19.2 回调与数据共享

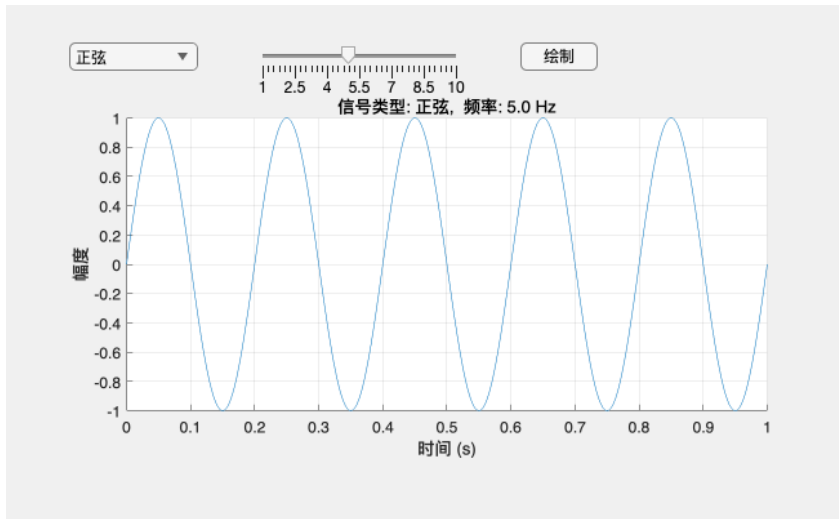
- 多个控件协同时，一个控件的回调常要读另一些控件的当前值。本节让“绘制”按钮（`uibutton`，回调 `ButtonPushedFcn`）按下时，读取下拉框（`uidropdown`）选的信号类型、滑块（`uislider`）调的频率。
- 读取控件的当前值用它的 `.Value` 属性。
- 控件间共享数据的正确做法是嵌套函数（回调作为嵌套函数，可直接访问父函数工作区里的控件句柄）或显式传递控件句柄；**不要用全局变量**（易冲突、难维护）。
- 这就是让界面状态在事件之间留存：连连看的棋盘、当前选中的块等状态，要被先后触发的多个回调读写，办法就是把它放在主函数工作区、由嵌套函数共享。

例 19.2

用纯代码 `uifigure` 建立一个界面：下拉框选信号类型（正弦或方波）、滑块调频率、按钮触发绘制；点按钮时按当前选择画出相应信号。

请用纯代码 `uifigure` 建界面：`uidropdown` 选信号类型（正弦/方波）、`uislider` 调频率（1~10 Hz）、`uibutton` “绘制”、`uiaxes`。
点按钮时读取下拉框与滑块当前值，画出对应类型与频率的信号；界面创建后立即画一次初始图。
请用嵌套函数或控件句柄共享数据，不要用全局变量。

例 19.2 运行结果·界面截图



例 19.2 核对

对结果的核验

- 选不同类型、不同频率再绘制，应画出对应的信号；正弦与方波形状不同、频率相符。截图为初始的正弦、5 Hz，0 ~ 1 秒内有 5 个完整周期。

对代码的核验

- 按钮回调写成嵌套函数，直接读下拉框 `dd.Value`、滑块 `sld.Value`，无需全局变量，数据共享方式合规。
- 回调读 `.Value` 取各控件当前状态；回调置于嵌套/文件末尾，位置合规。

易错点：模型常为简便而用全局变量

- 控件间共享数据有三种办法：嵌套函数（推荐）、`guidata`、全局变量（不推荐）。编码模型有时为简便而用全局变量，可追问“请改用嵌套函数或传递控件句柄共享数据，不要用全局变量”。

§19.3 信号时域/频谱查看器

- 把前两节合起来，建立一个实用工具，即信号时域/频谱查看器：下拉框选信号类型、滑块调频率，上方坐标轴看时域波形、下方坐标轴看幅度谱，改变参数时两轴实时更新。
- 计算频谱复用第 4 章写好的 `prefourier`：界面创建时调用一次得到变换矩阵 FT ，回调里用 $FT*y$ 求频谱再取幅度。`prefourier` 给出的 `omg` 是角频率 (rad/s)，画幅度谱时换算成 Hz。
- 频谱图上谱峰的位置就是信号的频率，就用这条信号知识检验界面对不对。

例 19.3 提示词

把前两节合起来，构造一个信号时域/频谱查看器：选信号类型、调频率，上方显示时域波形、下方显示幅度谱，改变参数时两轴实时更新。

请用纯代码 `uifigure` 建一个“信号时域/频谱查看器”：放一个 `uidropdown` 选择信号类型（正弦/方波/chirp）、一个 `uislider` 调节频率（1~100 Hz），上方一个 `uiaxes` 画时域波形、下方一个 `uiaxes` 画幅度谱；改变下拉框或滑块时两个坐标轴都实时更新；界面创建后立即画一次初始的时域波形与幅度谱。

幅度谱不要用 `fft` 计算，改用我已经写好的函数 `prefourier` 返回的傅里叶变换矩阵。请直接调用 `prefourier`、不要重新定义，函数代码如下：

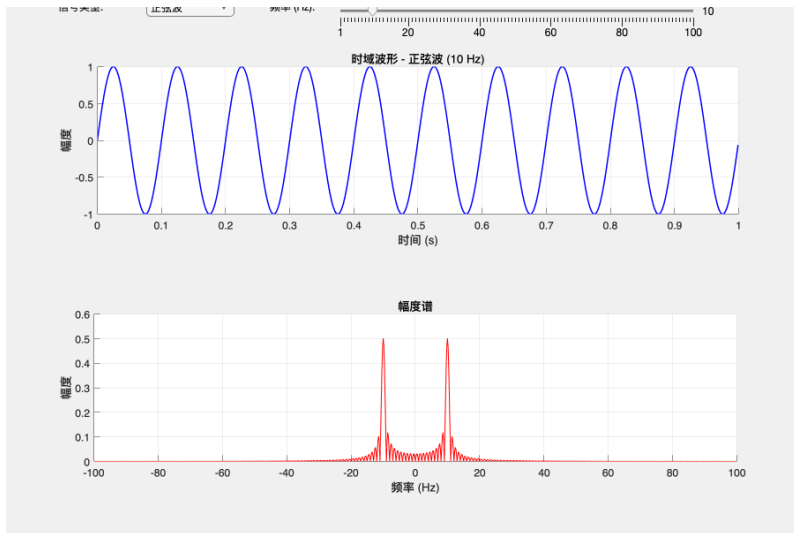
```
function [t, omg, FT, IFT] = prefourier(Trg, N, OMGrg, K)
.....
end
```

例 19.3 提示词 (续)

界面创建时先调用一次 `prefourier`: 时域范围取 0~1 秒、抽样点数 $N=1000$, 频域范围覆盖正负 100 Hz 对应的角频率、抽样点数 $K=1000$, 得到时间轴 t 、频率轴 omg 与变换矩阵 FT 。

回调里按当前信号类型与频率在 t 上生成信号 y (列向量), 用 $FT*y$ 得到频谱、在下方坐标轴画其幅度。`prefourier` 的 omg 是角频率 (rad/s), 画幅度谱时把横轴换算成 Hz。

例 19.3 运行结果·界面截图



例 19.3 核对

对结果的核验

- 幅度谱的峰值应落在所选频率处：截图为 10 Hz 正弦，双边谱的两个峰恰在 ± 10 Hz。
- 改变频率，谱峰随之移动；时域波形 1 秒内的周期数应等于频率（10 Hz 即 10 个周期）。
- 界面是否美观是一回事，谱峰位置对不对，要用频谱知识来判断。

对代码的核验

- 频谱由 `prefourier` 的变换矩阵左乘 $FT*y$ 求得，不用 `fft`；变换矩阵在界面创建时只算一次，回调里只做矩阵乘、不重复构造。
- `prefourier` 的 `omg` 是角频率 (rad/s)，画谱时须换算成 Hz ($omg/(2*\pi)$)；若忘了换算，谱峰会落在 $\pm 20\pi$ 而非 ± 10 Hz，相差 2π 倍。

例 19.3 延伸：从查看器到连连看

- 这个查看器把第 4 章的 `prefourier` 傅里叶变换矩阵包装成了可交互的工具：拖动滑块，就能看频谱随频率变化。
- 提示词要求“直接调用、不要重新定义”，模型却仍把 `prefourier` 附在文件末尾、自带一份。这样代码自包含、复制即能运行，定义也与第 4 章一致、并无错误，只是并非真正复用已有的函数。